

ActuaryGPT — Agentic AI Underwriting & Claims Triage Assistant

ActuaryGPT is an enterprise-grade AI-powered underwriting and claims triage system designed for insurance carriers. It features a complete multi-step applicant form portal, a machine learning risk classification engine, a Retrieval-Augmented Generation (RAG) database for case auditing, and a Gemini-powered conversational assistant to help underwriters make fast, accurate decisions.

■ Project Architecture Overview

```
graph TD
    subgraph Client_Portal [Frontend Portal - React/Vite]
        A[Customer View] -->|File Claim/Apply| C[Smart Multi-step Wizard]
        B[Officer View] -->|Audit Cases| D[Underwriter Dashboard]
        B -->|Triage Queue| E[AI Profiler Dashboard]
        B -->|Analytics| F[Interactive Charts]
        B -->|Co-pilot Chat| G[AI Assistant]
    end

    subgraph API_Service [FastAPI Backend]
        H[API Router]
        H -->|Auth| I[Database Connection]
        H -->|Underwriting / Triage| J[AI Underwriting Pipeline]
        H -->|Analytics Summary| K[Aggregations Engine]
        H -->|Co-Pilot| L[LLM Controller]
    end

    subgraph Database [SQLite Reference Store]
        M[(actuary_gpt.db)]
        M -->|User accounts| N[users]
        M -->|Life policies| O[applications]
        M -->|Claims history| P[vehicle_applications]
    end

    subgraph AI_Pipeline [Automated Auditing Engine]
        Q[OCR extraction] --> R[CatBoost Risk Model]
        R --> S[Cosine Similarity RAG]
        S --> T[Gemini Explanation Agent]
        T --> U[PDF Report Generator]
    end
```

end

C -->|Requests| H

D -->|Requests| H

E -->|Analyze/Decide| H

G -->|Prompt| H

H -->|Query/Commit| M

J -->|Run| AI Pipeline

L -->|Contextual LLM Chat| T

■ Key Features

■ Role-Based Portals

1. Policyholder / Client Portal

- **Smart Form Wizard:** Dynamic steps to submit Life Policy Applications or Vehicle Claims.
- **Document Auto-fill (OCR):** Upload medical invoices or vehicle photos/insurance documents to automatically parse data using Gemini.
- **Case History Tracking:** View the current status (approved, rejected, pending review) of submitted transactions.
- **Guardrailed AI Assistant:** Access customer support chatbot restricted from seeing internal ML metrics or risk classes.

2. Underwriting Officer Portal

- **Control Center Dashboard:** Real-time metrics showing total claims, approved rate, manual review queue, and average claim values.
 - **Claims Triage Queue:** An interactive list of all pending applications waiting for evaluation.
 - **Automated AI Profiler:** Performs verification, CatBoost prediction, similar claim lookups (RAG), and generates a PDF report.
 - **Manual Override Controls:** Direct options for underwriters to approve, reject, or adjust payout amounts.
 - **System Health Monitor:** Live tracking of backend services, SQLite connection, and ML model statuses.
 - **Dynamic Analytics Panel:** Custom dashboards displaying monthly trends, fraud rates, and distribution graphs.
 - **Co-Pilot AI Assistant:** Active chat interface with underwriting context integration.
-

■ ■ Technology Stack

Frontend (Client Application)

- **Framework:** React + Vite
- **Styling:** Vanilla CSS with a responsive layout, dark mode colors, glassmorphism, and keyframe animations.
- **Telemetry & Visualization:** Recharts (for charts), Lucide React (for icons)
- **API Integration:** Fetch API with dynamic hostname routing to switch between localhost and Render production URLs.

Backend (API Service)

- **Framework:** FastAPI (Python)
 - **Application Server:** Uvicorn
 - **Database:** SQLite 3 (with PostgreSQL fallback support)
 - **LLM Engine:** Google Gemini (using the modern `google.generativeai` or standard client)
 - **Machine Learning:** CatBoost + Scikit-Learn (for risk scoring and fraud classification)
 - **Report Generation:** ReportLab (for rendering pixel-perfect actuarial PDFs)
-

■ AI Underwriting & Claims Audit Pipeline

When an underwriter runs the **AI Profiler** on a claim, a 10-step auditing pipeline is executed in sequence:

1. **Document Validation:** Verifies structural parameters and flags discrepancies (e.g., age bounds).
 2. **OCR Extraction:** Runs character recognition on attached files (medical bills, crash reports) using multimodal Gemini models.
 3. **Applicant/Vehicle Verification:** Validates the policyholder's demographic profile against reference records.
 4. **Policy Verification:** Cross-references the requested payout or coverage limits with policy guidelines.
 5. **Damage / Health Risk Analysis:** Evaluates medical history details (smoker, BMI) or collision parameters (impact severity, collision type).
 6. **CatBoost ML Inference:**
 - **Life Model (`risk\_model.pkl`):** Classifies the health risk profile into classes 1 to 8.
 - **Vehicle Model (`vehicle\_claim\_model.pkl`):** Classifies fraud risk score and anomaly probabilities.
 7. **RAG Similar Case Retrieval (`similarity.py`):** Computes cosine similarity across historical records to retrieve the top 3 most similar resolved claims for context.
 8. **Premium / Claim Calculation:** Adjusts calculated premiums or recommends claim payouts.
 9. **Gemini Explanation Agent:** Synthesizes ML scores, similarity metrics, and OCR data to generate a human-readable case report.
 10. **PDF Report Generation (`report.py`):** Compiles the audit details into a formatted PDF document.
-

■ ■ Database Schema & Seeding

The database `actuary\_gpt.db` contains three core tables:

1. `users` Table

Tracks accounts and limits permissions:

- `username` (Text, Unique Primary Key)
- `password` (Text)
- `role` (Text - `customer` or `officer`)

2. `applications` Table

Stores Life & Health policy applications:

- `id` (Text, Primary Key)
- `client` (Text, Foreign Key)
- `age`, `height`, `weight`, `bmi`
- `occupation`, `income`
- `smoker` (Integer), `previous\_claims` (Integer), `family\_history` (Integer)
- `insurance\_type` (`Life` or `Health`)
- `coverage\_amount`, `premium`
- `status` (`pending`, `approved`, `rejected`)
- `risk\_class` (Integer), `risk\_category` (Text)
- `report` (Text), `pdf\_url` (Text)

3. `vehicle\_applications` Table

Stores vehicle claims and fraud audits:

- `id` (Text, Primary Key)
- `client` (Text, Foreign Key)
- `months\_as\_customer`, `age`
- `policy\_state`, `policy\_csl`, `policy\_deductable`, `policy\_annual\_premium`, `umbrella\_limit`
- `insured\_sex`, `insured\_education\_level`, `insured\_occupation`, `insured\_hobbies`, `insured\_relationship`
- `capital\_gains`, `capital\_loss`
- `incident\_type`, `collision\_type`, `incident\_severity`, `authorities\_contacted`, `incident\_state`, `incident\_city`
- `incident\_hour\_of\_the\_day`, `number\_of\_vehicles\_involved`, `property\_damage`, `bodily\_injuries`,
`witnesses`, `police\_report\_available`
- `total\_claim\_amount`, `injury\_claim`, `property\_claim`, `vehicle\_claim`
- `auto\_make`, `auto\_model`, `auto\_year`
- `status` (`pending`, `approved`, `rejected`)
- `fraud\_reported` (`Y` or `N`)

Database Initial Seeding

When initialized, `database.py` automatically:

1. Seeds default credentials (`actuary1` and `customer1`).
2. Populates `applications` with sample life data.
3. Parses historical insurance claims data from the `datasets/insurance\_claims.csv` dataset and inserts the records into `vehicle\_applications`.

■ API Endpoints Reference

| Category | Endpoint | Method | Description |

| :--- | :--- | :--- | :--- |

| **Auth** | `/auth/register` | `POST` | Registers a new customer |

| | `/auth/login` | `POST` | Authenticates users (returns role & username) |

| **Life** | `/applications` | `GET` | Fetches Life applications (can filter by status/client) |

| | `/applications` | `POST` | Submits a new Life policy application |

| | `/applications/{id}/evaluate` | `POST` | Runs the automated ML & Gemini auditing pipeline |

| | `/applications/{id}/decide` | `POST` | Submits approval, rejection, or manual review decision |

| **Vehicle** | `/applications/vehicle` | `GET` | Fetches Vehicle claims |

| | `/applications/vehicle` | `POST` | Submits a new Vehicle claim |

| | `/applications/vehicle/{id}/evaluate` | `POST` | Runs the claims fraud triage pipeline |

| | `/applications/vehicle/{id}/decide` | `POST` | Registers payout decision or overrides payout |

| **Analytics** | `/analytics/summary` | `GET` | Returns aggregated metrics, distribution data, and processed lists |

| **Chatbot** | `/chatbot` | `POST` | Interface with Gemini underwriting/customer assistant |

■ Installation & Running Locally

Prerequisites

- Python 3.10+
- Node.js 18+

Step 1: Clone and Set Up Backend

1. Navigate to the backend directory:

```
cd backend
```

2. Create a virtual environment and activate it:

```
python -m venv venv

# On Windows:

venv\Scripts\activate

# On Linux/macOS:

source venv/bin/activate
```

3. Install the dependencies:

```
pip install -r requirements.txt
```

4. Create a `.env` file in the ``backend/app/`` folder and add your Gemini API Key:

```
GEMINI_API_KEY=your_google_gemini_api_key_here
```

5. Run the server using Uvicorn:

```
python -m uvicorn backend.app.main:app --host 127.0.0.1 --port 8000
```

Step 2: Set Up Frontend

1. Navigate to the frontend directory:

```
cd ../frontend
```

2. Install dependencies:

```
npm install
```

3. Start the Vite dev server:

```
npm run dev
```

4. Open `[http://localhost:5173/](http://localhost:5173/)` in your web browser.

■ Deployment Details

The application is configured to deploy directly to the cloud via standard workflows:

1. **Frontend Hosting:** Deployed on **Vercel** (``actuarygpt-5h5s.vercel.app``).
2. **Backend API Hosting:** Deployed on **Render** (``actuarygpt-backend.onrender.com``).

Dynamic API Base Switching

In `App.jsx`, the frontend automatically switches the API connection parameters to avoid browser CORS/Mixed Content security blocks:

```
const API_BASE = window.location.hostname === "localhost" || window.location.hostname === "127.0.0.1"

  ? "http://127.0.0.1:8000"

  : "https://actuarygpt-backend.onrender.com";
```

This guarantees a clean local testing sandbox on port 5173 while remaining fully operational on the web!